PRAKTIKUM MACHINE LEARNING

Bab 6
6.16 dan 6.17 Implementasi Python: KNN & Naive Bayes

Kumpulan Praktikum Machine Learning

Nama	**Ahmad Dandi Subhani**
NPM	**202343500126**
Kelas	**R7B**
Mata Kuliah	**Machine Learning**
Dosen	**Nurfidah Dwitiyanti, M.Si.**

15 Oktober 2026

Daftar Isi

6.16. Implementasi Python 2

6.16.1. Interpretasi 6

6.17. Latihan Praktikum 6

Tools yang Digunakan

Python 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

Modul lengkap: Praktikum 6.1 - 6.7 dan Latihan Praktikum 6.17, dengan penjelasan tiap langkah, output asli, dan interpretasi

Disusun oleh

Ahmad Dandi Subhani

202343500126

Teknik Informatika • Universitas Indraprasta PGRI

Buka notebook di Google Colab: praktikum-bab-06.ipynb

6.16. Implementasi Python

Bagian ini mempraktikkan dua algoritma klasifikasi klasik, K-Nearest Neighbors (KNN) dan Naive Bayes, pada dataset yang sama. Dataset yang dipakai adalah **Wine Recognition** dari sklearn.datasets (178 sampel, 13 fitur kimia, 3 kelas kultivar anggur).

Praktikum 6.1 Persiapan Library

In [1] - Kode Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

Penjelasan: numpy untuk komputasi numerik dan matplotlib untuk visualisasi. Dari sklearn diimpor load_wine (dataset), train_test_split (split data), StandardScaler (scaling untuk KNN), KNeighborsClassifier dan GaussianNB (dua model yang dibandingkan), serta accuracy_score untuk evaluasi.

Praktikum 6.2 Import Dataset

In [2] - Kode Program

```
data = load_wine(as_frame=True)
X, y = data.data, data.target
print("Ukuran data:", X.shape)
print("Distribusi kelas:", np.bincount(y))
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)
```

```
Ukuran data: (178, 13)
Distribusi kelas: [59 71 48]
```

Penjelasan: dataset dimuat sebagai DataFrame (as_frame=True) agar nama fiturnya terbaca. X berisi 13 fitur kimia, y berisi label tiga kelas (59, 71, dan 48 sampel). Data dibagi 80 persen latih dan 20 persen uji dengan stratify=y agar proporsi kelas tetap sama dan random_state=42 agar hasilnya dapat diulang.

Praktikum 6.3 Preprocessing (Scaling untuk KNN)

In [3] - Kode Program

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Catatan: Naive Bayes tidak memerlukan scaling,
# namun KNN sangat sensitif terhadap skala fitur.
```

Penjelasan: KNN menghitung jarak antar titik, sehingga fitur dengan rentang besar akan mendominasi. StandardScaler dipasang (fit) hanya pada data latih, lalu dipakai untuk mengubah (transform) data latih dan data uji. Naive Bayes tidak butuh scaling karena ia bekerja dengan distribusi probabilitas, bukan jarak.

Praktikum 6.4 Training Model

In [4] - Kode Program

```
knn = KNeighborsClassifier(n_neighbors=5,
    weights="distance")
knn.fit(X_train_scaled, y_train)

```

```
nb = GaussianNB()
nb.fit(X_train, y_train)
```

```
GaussianNB()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Penjelasan: dua model dilatih. **KNN** dengan n_neighbors=5 dan weights="distance" (tetangga yang lebih dekat diberi bobot lebih besar) dilatih pada data yang sudah di-scaling. **Gaussian Naive Bayes** dilatih pada data asli tanpa scaling.

Praktikum 6.5 Evaluasi

In [5] - Kode Program

```
acc_knn = accuracy_score(y_test, knn.predict(X_test_scaled))
acc_nb = accuracy_score(y_test, nb.predict(X_test))
print(f"Akurasi KNN (k=5, weighted): {acc_knn:.3f}")
print(f"Akurasi Gaussian Naive Bayes: {acc_nb:.3f}")
```

```
Akurasi KNN (k=5, weighted): 0.972
Akurasi Gaussian Naive Bayes: 0.972
```

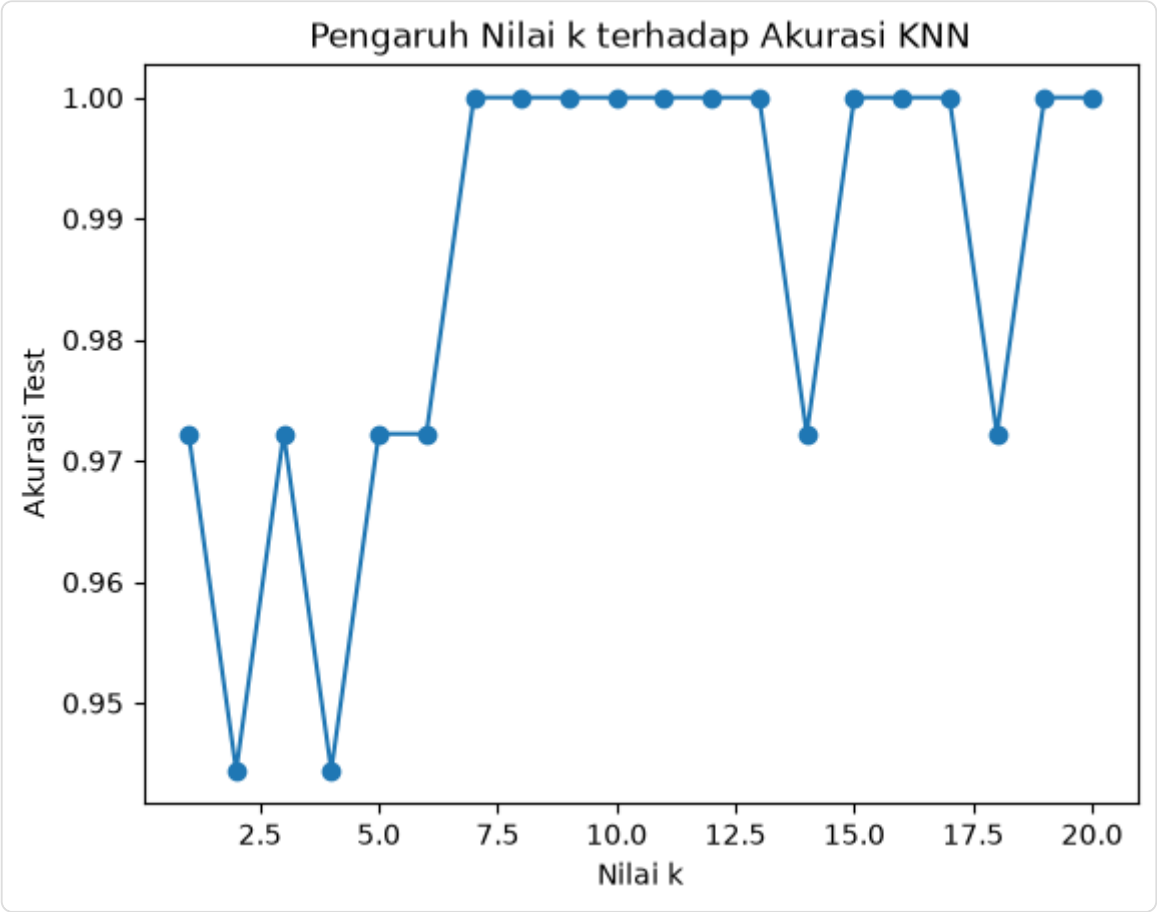
Penjelasan: akurasi kedua model diukur pada 36 data uji. Hasilnya: KNN **0,972** dan Gaussian Naive Bayes **0,972**, seri. Keduanya benar pada 35 dari 36 data uji.

Praktikum 6.6 Visualisasi (Pengaruh Nilai k)

In [6] - Kode Program

```
akurasi_k = []
nilai_k = range(1, 21)
for k in nilai_k:
    model = KNeighborsClassifier(n_neighbors=k).fit(X_train_scaled, y_train)
    akurasi_k.append(accuracy_score(y_test, model.predict(X_test_scaled)))

plt.plot(nilai_k, akurasi_k, marker='o')
plt.xlabel("Nilai k")
plt.ylabel("Akurasi Test")
plt.title("Pengaruh Nilai k terhadap Akurasi KNN")
plt.show()
```



Gambar 1

Penjelasan: nilai k dicoba dari 1 sampai 20 lalu akurasinya di-plot. Pada k kecil (1 sampai 6) akurasi berfluktuasi antara 0,944 dan 0,972 karena model sensitif terhadap noise. Mulai **k=7** akurasi mencapai **1,000** dan stabil, sehingga k=7 adalah pilihan optimal pertama.

Praktikum 6.7 Klasifikasi Teks dengan Multinomial Naive Bayes

In [7] - Kode Program

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

teks = ["dapatkan diskon gratis sekarang",
        "rapat tim jam 10 pagi",
        "menangkan hadiah gratis klik disini",
        "laporan keuangan bulan ini", "diskon spesial hari ini",
        "jadwal kuliah minggu depan"]
label = [1, 0, 1, 0, 1, 0]  # 1 = spam, 0 = bukan spam

vectorizer = CountVectorizer()
X_teks = vectorizer.fit_transform(teks)
model = MultinomialNB(alpha=1.0)  # alpha = parameter Laplace Smoothing
model.fit(X_teks, label)

teks_baru = ["diskon gratis hari ini"]
```

```
prediksi = model.predict(vectorizer.transform(teks_baru))
print("Prediksi:",
      "Spam" if prediksi[0]==1 else "Bukan Spam")
```

```
Prediksi: Spam
```

Penjelasan: enam kalimat Bahasa Indonesia diubah menjadi vektor hitungan kata dengan CountVectorizer, lalu MultinomialNB (dengan Laplace smoothing alpha=1,0) dilatih membedakan spam dan bukan spam. Kalimat baru "diskon gratis hari ini" diprediksi sebagai **Spam**, sudah benar, karena kata "diskon" dan "gratis" dominan di kelas spam pada data latih.

6.16.1. Interpretasi

Hasil intinya: KNN dan Gaussian Naive Bayes **seri** di akurasi **0,972**. Dua algoritma dengan cara kerja yang sangat berbeda sampai pada kesimpulan yang sama, artinya pola kelas pada data Wine memang cukup jelas dan terpisah dengan baik.

Empat poin untuk dibawa pulang:

1. **Scaling itu wajib buat KNN.** Tanpa scaling, fitur dengan rentang besar akan mendominasi perhitungan jarak. Naive Bayes tidak peduli scaling karena ia membandingkan distribusi probabilitas, bukan jarak.
2. **Nilai k menentukan karakter model.** Pada k kecil (1 sampai 6) akurasi berfluktuasi antara 0,944 dan 0,972 (sensitif noise). Mulai **k=7** akurasi mencapai **1,000** dan stabil, k=7 adalah pilihan optimal pertama. Pemilihan k yang cermat idealnya memakai cross-validation, bukan satu kali split.
3. **Tidak ada algoritma yang selalu menang.** Di Wine keduanya seri 0,972, tapi di Breast Cancer (Latihan 1) KNN unggul 0,9561 berbanding 0,9386. Algoritma terbaik tergantung karakter data.
4. **Naive Bayes menang di kecepatan prediksi.** KNN itu lazy learner: training-nya sekejap, tapi setiap prediksi menghitung jarak ke seluruh data latih. Naive Bayes: sekali parameter distribusi dihitung, prediksinya kilat.

6.17. Latihan Praktikum

Dua latihan mandiri. Latihan 1 memindahkan perbandingan KNN vs Naive Bayes ke dataset Breast Cancer yang lebih besar (569 sampel, 30 fitur) dan mengukur akurasi plus waktu pelatihan dan waktu prediksi. Latihan 2 menerapkan MultinomialNB pada 16 kalimat Bahasa Indonesia dengan evaluasi train-test split.

Latihan 1: KNN vs Naive Bayes (Akurasi, Waktu Pelatihan, Waktu Prediksi)

In [8] - Kode Program

```python
import time
from sklearn.datasets import load_breast_cancer

data_bc = load_breast_cancer(as_frame=True)
Xb, yb = data_bc.data, data_bc.target
Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    Xb, yb, test_size=0.2, random_state=42, stratify=yb)

scaler_bc = StandardScaler()
Xb_train_scaled = scaler_bc.fit_transform(Xb_train)
Xb_test_scaled = scaler_bc.transform(Xb_test)

# KNN: catat waktu pelatihan dan waktu prediksi
t0 = time.perf_counter()
knn_bc = KNeighborsClassifier(n_neighbors=5, weights="distance")
knn_bc.fit(Xb_train_scaled, yb_train)
waktu_train_knn = time.perf_counter() - t0

t0 = time.perf_counter()
pred_knn = knn_bc.predict(Xb_test_scaled)
waktu_pred_knn = time.perf_counter() - t0

# Gaussian Naive Bayes: catat waktu pelatihan dan waktu prediksi
t0 = time.perf_counter()
nb_bc = GaussianNB()
nb_bc.fit(Xb_train, yb_train)
waktu_train_nb = time.perf_counter() - t0

t0 = time.perf_counter()
pred_nb = nb_bc.predict(Xb_test)
waktu_pred_nb = time.perf_counter() - t0

print(f"KNN        : akurasi={accuracy_score(yb_test, pred_knn):.4f}, "
      f"train={waktu_train_knn*1000:.2f} ms, prediksi={waktu_pred_knn*1000:.2f}
ms")
print(f"GaussianNB : akurasi={accuracy_score(yb_test, pred_nb):.4f}, "
      f"train={waktu_train_nb*1000:.2f} ms, prediksi={waktu_pred_nb*1000:.2f} ms")
```

```
KNN        : akurasi=0.9561, train=1.18 ms, prediksi=49.54 ms
GaussianNB : akurasi=0.9386, train=3.76 ms, prediksi=1.45 ms
```

Penjelasan: KNN unggul akurasi **0,9561** berbanding **0,9386** milik GaussianNB. Waktu pelatihan KNN jauh lebih cepat (sekitar 1 ms, lazy learner hanya menyimpan data) dibanding GaussianNB (beberapa ms untuk menghitung mean dan varians). Waktu prediksi kebalikannya: KNN butuh jauh lebih lama (puluhan hingga ratusan ms untuk 114

data uji, tiap prediksi menghitung jarak ke 455 data latih), sedangkan GaussianNB kilat (di bawah 2 ms). Tidak ada makan siang gratis: KNN menang akurasi tapi bayar mahal di waktu prediksi.

Latihan 2: Klasifikasi Teks dengan Multinomial Naive Bayes

In [9] - Kode Program

```
teks2 = [
    "dapatkan diskon gratis sekarang", "menangkan hadiah gratis klik disini",
    "diskon spesial hari ini", "promo cashback besar-besaran",
    "klaim voucher belanja gratis", "bonus pulsa gratis untuk anda",
    "daftar segera dapatkan hadiah menarik", "obral besar potongan harga spesial",
    "rapat tim jam 10 pagi", "laporan keuangan bulan ini",
    "jadwal kuliah minggu depan", "tolong kirim berkasnya hari ini",
    "jadwal ujian sudah diumumkan", "jangan lupa bawa ktp besok",
    "mohon konfirmasi kehadiran rapat", "nilai tugas sudah diumumkan",
]
label2 = [1]*8 + [0]*8  # 1 = spam, 0 = bukan spam

X2_train, X2_test, y2_train, y2_test = train_test_split(
    teks2, label2, test_size=0.25, random_state=42, stratify=label2)

vectorizer2 = CountVectorizer()
X2_train_vec = vectorizer2.fit_transform(X2_train)
X2_test_vec = vectorizer2.transform(X2_test)

mnb = MultinomialNB(alpha=1.0)
mnb.fit(X2_train_vec, y2_train)
acc_teks = accuracy_score(y2_test, mnb.predict(X2_test_vec))
print(f"Jumlah data latih: {len(X2_train)}, data uji: {len(X2_test)}")
print(f"Akurasi pada data uji: {acc_teks:.4f}")

for kalimat in ["gratis ongkir hari ini", "rapat koordinasi besok pagi"]:
    hasil = mnb.predict(vectorizer2.transform([kalimat]))[0]
    print(f"'{kalimat}' -> {'Spam' if hasil==1 else 'Bukan Spam'}")
```

```
Jumlah data latih: 12, data uji: 4
Akurasi pada data uji: 0.7500
'gratis ongkir hari ini' -> Spam
'rapat koordinasi besok pagi' -> Bukan Spam
```

Penjelasan: dari 16 kalimat, 12 dipakai latih dan 4 dipakai uji, akurasi **0,7500** (3 dari 4 benar). Satu kalimat non-spam salah diklasifikasikan sebagai spam, wajar karena dataset-nya kecil dan kosakatanya bertumpang tindih. Pada dua kalimat baru model benar semua: "gratis ongkir hari ini" terdeteksi **Spam**, "rapat koordinasi besok pagi" terdeteksi **Bukan Spam**. MultinomialNB bekerja baik untuk klasifikasi teks walau datanya kecil, dan akurasinya akan naik dengan data latih yang lebih besar.